

SE 413/513

Semester Project Guidelines and Requirements

Fall 2026

This document supplements the SE 413/513 course syllabus and governs the semester software-testing project referenced in the “Semester Project” and “Grading Distribution” sections of the syllabus (Team software-testing project: 30%; Final presentation/advanced analysis: 10%). All syllabus policies (academic honesty, late work, generative-AI use, individual work) apply here unless explicitly modified below.

1. Overview

Each team will complete an end-to-end software-testing project, applying the test-design, automation, and evaluation techniques covered in class (unit, control-flow, data-flow, coverage-based, functional, integration, system, and acceptance testing) to a system of realistic (but appropriately bounded) size and complexity.

2. Team Formation

- Projects are completed **individually by default**. A team of **two students** is permitted with instructor approval, provided the proposed scope in the specification document is expanded accordingly to justify two contributors.
- By the **end of Week 3**, each student must notify the instructor via Blackboard whether they are working individually or requesting a two-person team, and if the latter, with whom.
- For two-person teams, both members must submit a brief **Individual Contribution Statement** with every written deliverable (progress report and final report), describing each member’s specific responsibilities and work product. Grades may be adjusted individually within a team if contributions are materially uneven.

3. Project Track Selection

Each student/team selects **one** of the two tracks below and confirms the choice in the Week 4 specification document.

3.1 Track A – Testing an Existing System

Select an existing, actively maintained open-source project (preferably written in Python) that currently has thin or absent automated tests. The project’s deliverable is a substantially expanded, well-organized automated test suite plus supporting testing documentation (test plan, coverage report, defect log).

3.2 Track B – Testing a Newly-Developed System

Design and implement a small original system, preferably in Python (a REST API, CLI tool, or similar), then test it end-to-end.

3.3 Approval

The instructor will review each specification document within one week of the Week 4 deadline and will approve the proposed system/scope or request a revision. Development beyond basic setup should not begin in earnest until the track and scope are approved.

4. GitHub Repository Requirements

- Each student/team must create a GitHub repository for the project (for Track A, a fork of the existing project) and add the instructor as a collaborator **no later than the Week 4 specification deadline**. The repository URL and the instructor's GitHub username (mdkamrulhasan) used for the invitation must both be included in the specification document.
- Development must be tracked through the repository's commit history. Commits should be **regular and incremental** across the project timeline, with descriptive messages – a small number of large, late commits will be treated as a red flag during grading regardless of the final code quality.
- The repository must include a README describing the system, how to install dependencies, and how to run the test suite.
- Consistent with the Week 13 “Continuous Testing” topic, teams are strongly encouraged (though not strictly required unless stated otherwise in class) to configure a GitHub Actions workflow that runs the test suite and reports coverage automatically on each push.
- If the repository location changes after Week 4, notify the instructor immediately.

5. Milestones and Deliverables

When	Deliverable	What's Due
End of Week 3	Team/track intent	Notify instructor: individual or two-person team (and with whom)
End of Week 4	Project Specification Document	Full spec per Section 5.1; GitHub repo created and shared
Weeks 7–9	Progress Report #1 + Check-In	Written progress report (Section 5.2) plus a brief 20–30 min meeting with the instructor (Schedule a 30-minute meeting)
Weeks 13–14	Final Project Report	Full final report per Section 5.3
Weeks 13–14	Final Presentation	30-minute Zoom presentation per Section 5.4

5.1 Project Specification Document (Due End of Week 4)

Submit a single document (PDF) containing:

1. **Team & Track:** Names of team member(s); selected track (A or B); GitHub repository URL and collaborator invitation confirmation.
2. **System Description:** What the system does, its main components, and (for Track A) a link to the original upstream project and its current test-coverage status.
3. **Testing Objectives and Scope:** Which levels of testing will be performed (unit / integration / system / acceptance) and which parts of the system each will target.
4. **Test Plan Outline:** Planned test-design techniques (e.g., equivalence partitioning, boundary-value analysis, control-flow/data-flow criteria), tools (pytest, coverage.py, etc.), and a target code-coverage goal.
5. **Risk Assessment:** Two to three specific risks to completing the project on time (e.g., unfamiliar codebase, flaky external dependencies) and how each will be mitigated.
6. **Timeline:** A week-by-week plan from Week 4 through Week 14, mapped to the course schedule's testing topics where relevant.
7. **Deliverables Checklist:** Confirmation of what will be produced by the final report (test suite, coverage report, defect log, CI configuration, etc.).

5.2 Progress Report #1 (Weeks 7–9)

A short (1–2 page) written report submitted via Blackboard, followed by a brief 20–30 minute check-in meeting (Zoom or during office hours) with the instructor. The report should cover:

- Work completed against the Week 4 timeline, with a link to current test results/coverage in the GitHub repository.
- Any deviations from the original plan or scope, and why.
- Remaining risks or blockers, and the plan to address them before Week 13.

Teams that are significantly behind schedule at this checkpoint should expect a scope adjustment discussion with the instructor.

5.3 Final Project Report (Due Weeks 13–14)

A comprehensive written report (submitted as PDF, with the GitHub repository as supporting evidence) including:

- Final system/testing scope actually completed (noting any changes from the specification).
- Test design summary: techniques applied and representative examples of test cases for each testing level performed.
- Test execution results: pass/fail summary, code coverage achieved (with a coverage report), and how coverage was measured.

- Defect log: defects found, their severity, and resolution status.
- CI/automation summary, if configured.
- Findings and recommendations: an assessment of the system’s overall quality and reliability based on testing results.
- Lessons learned and, for two-person teams, the Individual Contribution Statement.
- AI-Assistance Disclosure (see Section 7).

5.4 Final Presentation (Weeks 13–14)

A 30-minute Zoom presentation to the class covering the system tested, testing approach, key results, and lessons learned, with time for questions. Slides are required and should be submitted alongside the final report.

6. Grading

The project maps onto the syllabus’s “Team software-testing project” (30%) and “Final presentation/advanced analysis” (10%) categories as follows. Exact rubrics for each component will be posted on Blackboard.

Component	Weight (of total grade)
Project Specification Document (Wk 4)	2%
GitHub Repository & Development Process (commit history, CI, organization)	9%
Progress Report #1 & Check-In (Wk 7–9)	4%
Final Project Report (Wk 13–14)	5%
Subtotal – Team software-testing project	20%
Final Presentation (Wk 13–14, 30-min Zoom/in-Class)	10%
Total tied to the semester project	30%

6.1 Late Policy for Project Deliverables

The syllabus late policy (10% of the maximum possible grade deducted per day late, up to three days) applies to the specification document, progress report, and final report. Because most projects are completed individually, this penalty applies directly to that student; for a two-person team submitting jointly, the penalty applies to both members unless the Individual Contribution Statement shows the delay was attributable to one member only, in which case the instructor may apply the penalty individually.

7. Use of Generative AI Tools in the Project

Consistent with the syllabus’s general AI-use policy, generative AI tools may assist with the project (e.g., boilerplate test scaffolding, debugging suggestions), but students remain fully responsible for understanding, validating, and being able to explain every line of test and application code

submitted. Each written deliverable (progress report and final report) must include a short **AI-Assistance Disclosure** listing which tools were used and for which specific parts of the work (e.g., “GitHub Copilot used for initial pytest fixture boilerplate; all assertions and edge cases written and verified manually”).

8. Appendix: Sample Project Specification Document

The following is a fully worked, illustrative example of a Week-4 specification document for a **single student** on **Track B** (newly-developed system), scoped to be realistically achievable in 6–8 weeks alongside a normal course load. It is provided as a model for structure, level of detail, and appropriate scope – not as a topic students must use.

Project Specification: Personal Expense Tracker Web Application

Student: Jane Doe **Track:** B (Newly-Developed System) **Team size:** 1

GitHub Repository: <https://github.com/janedoe/expense-tracker> (instructor added as collaborator: @mdkamrulhasan)

1. *System Description.* A Streamlit + SQLite web application for tracking personal expenses. Core features: user registration/login, CRUD operations on expense records, expense categories, a monthly summary/dashboard showing totals by category, and configurable monthly budget limits with an over-budget indicator. Estimated size: ~ 700–900 lines of application code.

2. *Testing Objectives and Scope.*

- **Unit testing:** Business-logic functions in isolation – category totals, budget-limit calculations, date-range filtering, and input validation.
- **Component testing:** Individual Streamlit application components and functions, including form inputs, expense-entry workflows, filtering, and summary calculations.
- **System testing:** End-to-end behavior of the Streamlit application using Streamlit’s testing framework, including user interactions, form submission, authentication, error handling, and dashboard updates.
- **Acceptance testing:** Three user stories (“As a user, I can register and log in,” “As a user, I can log an expense and see it reflected in my monthly summary,” “As a user, I am warned when I exceed my monthly budget”) verified end-to-end.

3. *Test Plan Outline.* Equivalence partitioning and boundary-value analysis for input validation (amounts, dates, and budget limits); decision-table testing for the budget-alert logic (under/at/over budget, multiple categories); pytest as the test runner; Streamlit’s AppTest framework for testing application interactions and UI behavior; and pytest-cov for coverage measurement. **Coverage goal:** 85% line coverage on application code (excluding configuration and environment-specific code).

4. *Risk Assessment.*

- **Risk:** Limited prior experience with Streamlit. *Mitigation:* Complete a focused Streamlit tutorial in Week 4–5 before core development; keep the application workflow small.
- **Risk:** Scope creep (adding features instead of testing them). *Mitigation:* Freeze feature scope after Week 6; any new feature ideas are logged but deferred.
- **Risk:** Underestimating time needed for testing application state and database interactions. *Mitigation:* Separate business logic from Streamlit UI code; use an isolated SQLite test database and test the application through AppTest.

5. *Timeline.*

- **Weeks 4–5:** Finalize data schema and application design; scaffold Streamlit app; set up GitHub repository and CI skeleton.
- **Weeks 5–6:** Implement core functionality (authentication, expense CRUD, and categories).
- **Week 6:** Implement monthly summary and budget-alert functionality; feature freeze.
- **Weeks 7–8:** Write unit tests for business logic; begin Streamlit component/application tests. (Progress Report #1 due within this window.)
- **Weeks 9–10:** Complete application and system tests; configure GitHub Actions CI to run the test suite on push.
- **Week 11:** Acceptance tests for the three user stories; measure and improve coverage toward the 85% goal.
- **Week 12:** Defect triage and fixes; finalize coverage report and defect log.
- **Week 13:** Draft final report; prepare presentation slides.
- **Week 14:** Submit final report; deliver 30-minute Zoom presentation.

6. *Deliverables Checklist.* Automated test suite (pytest) covering unit, component, system, and acceptance testing; Streamlit application tests using AppTest; coverage report ($\geq 85\%$ target); defect log; GitHub Actions CI configuration; final report with results and lessons learned.